Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to [0,1].
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

8. Source Code

The complete implementation of this experiment, including data preprocessing, MLP model development, hyperparameter optimization using `RandomizedSearchCV`, model evaluation, visualization scripts, and experimental results, is publicly available on GitHub.

GitHub Repository:

`https://github.com/PandiKabilesh2006/CS3807-Deep-Learning-Laboratory/tree/main/Lab%202%20Multi%20Layer%20Perceptron`

The repository contains the following files:

- **MLP.ipynb** – Complete Jupyter Notebook implementation.
- **README.md** – Project overview and execution instructions.
- **Figures/** – Training curves, confusion matrix, and result plots.
- **Report.pdf** – Laboratory report.

9. Mandatory Plots

9.1 Sample Images

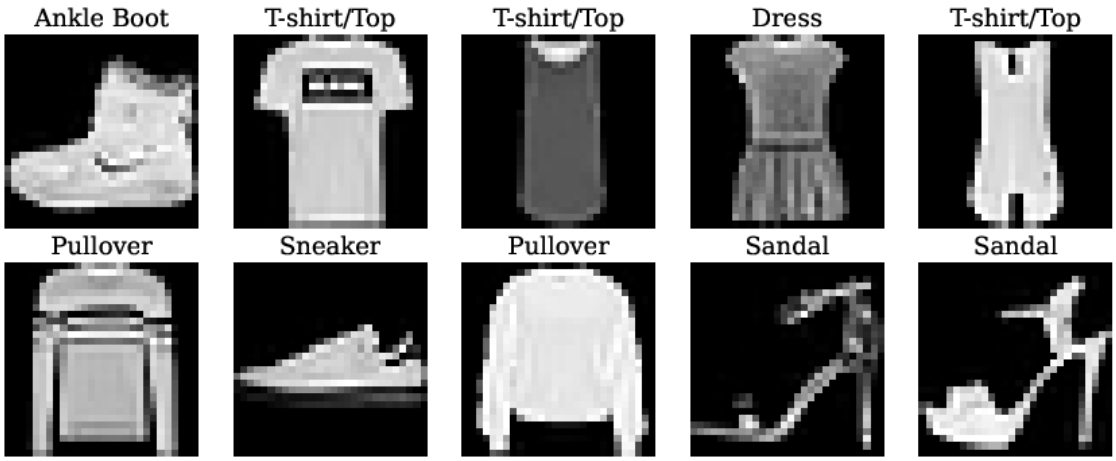

Figure 1: Sample Images from the Fashion-MNIST Dataset

Inference: The grid of samples shows that the dataset includes low-resolution grayscale pictures of apparel items (t-shirts, trousers, pullovers, sandals). Given the similarity of certain types of clothing (shirt vs. coat, sneaker vs. sandal), there will be confusion among different classes when classifying the objects.

9.2 Class Distribution

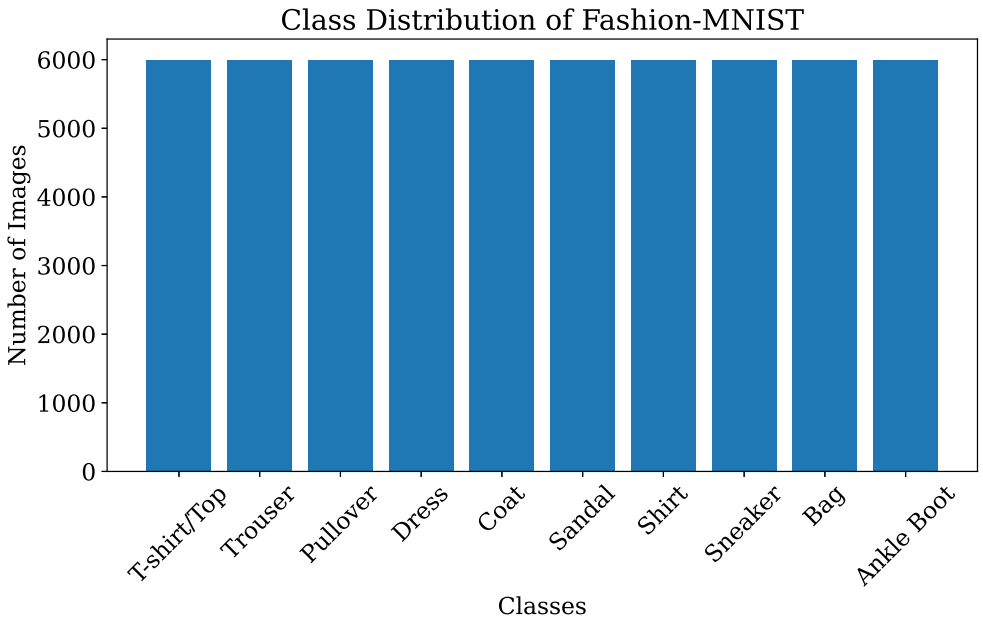

Figure 2: Class Distribution of Training Samples

Inference: All 10 classes have an equal (or almost equal) number of instances. It is thus clear that Fashion MNIST is a balanced dataset. In such datasets, accuracy is a suitable measure to use for evaluating the performance of a model.

9.3 Training Accuracy vs Epoch

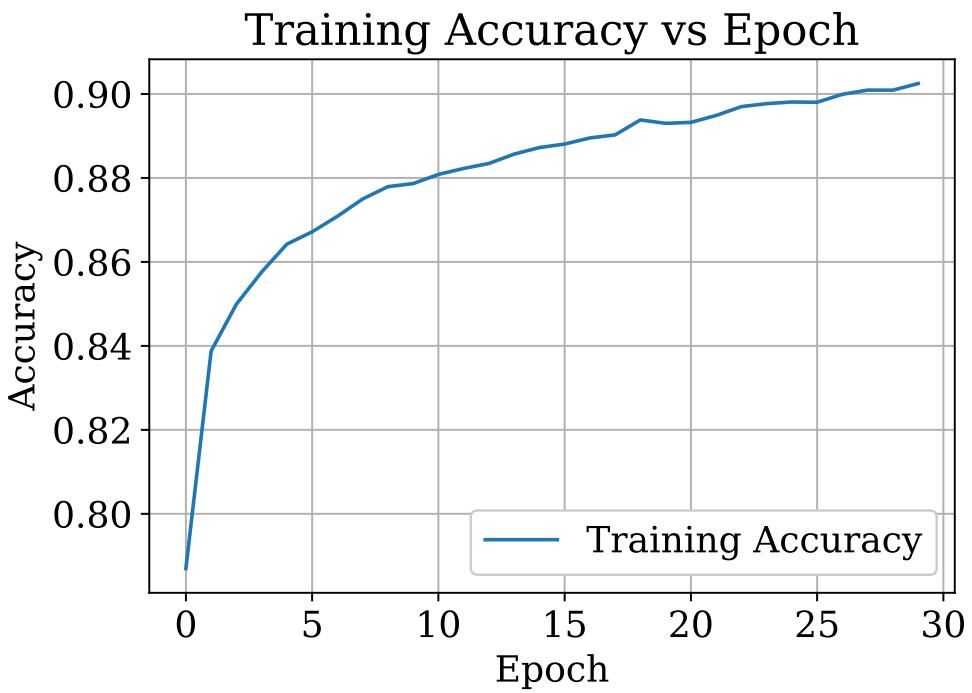

Figure 3: Training Accuracy vs Epoch

Inference: The accuracy of training increases rapidly during the first epochs and then increases at a slower rate after that, which shows that the neural network is continuously learning discriminative features from the flat vectors of pixels. The steady rise without any drastic fall in the curve shows consistent updating of gradients using Adam optimizer.

9.4 Validation Accuracy vs Epoch

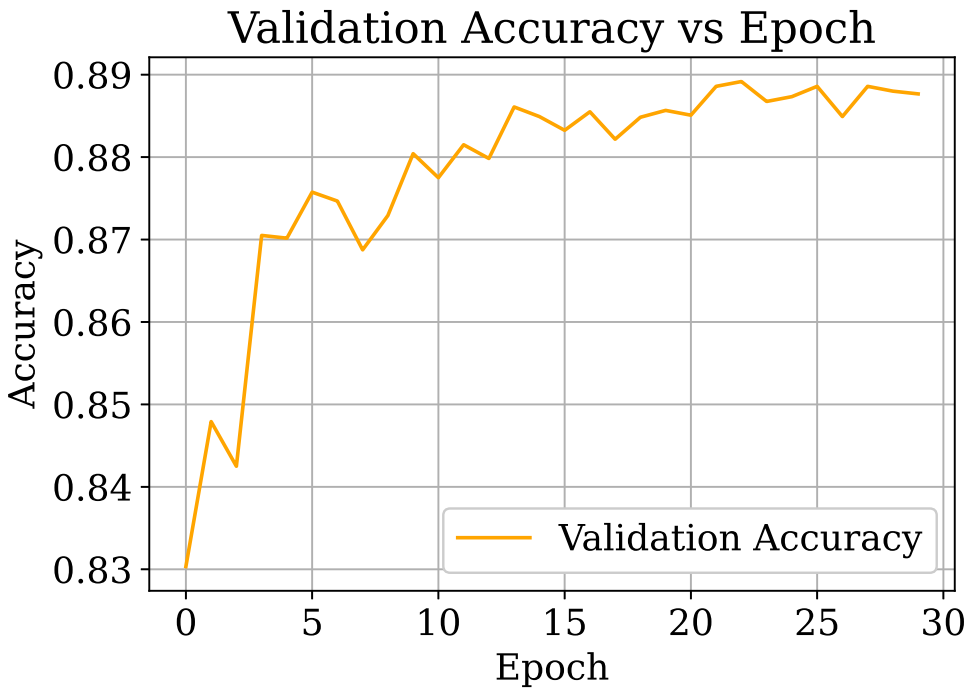

Figure 4: Validation Accuracy vs Epoch

Inference: The accuracy of validation is very close to that of training throughout most of the epochs. In case where the validation accuracy levels off or drops slightly while the training accuracy continues to increase in the later epochs, it indicates the start of overfitting.

9.5 Training Loss vs Epoch

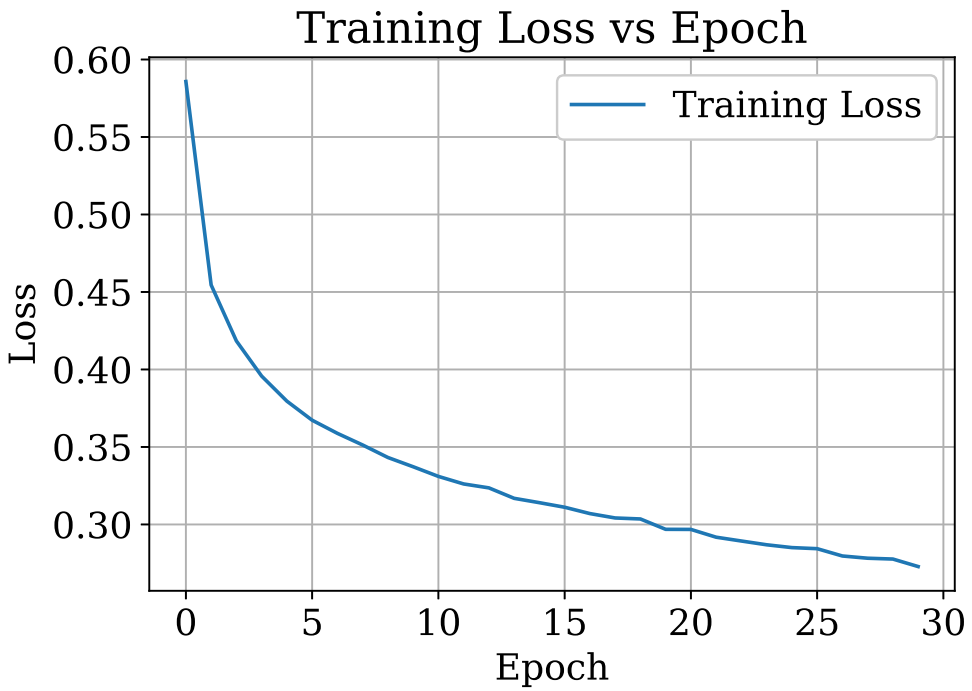

Figure 5: Training Loss vs Epoch

Inference: Categorical cross entropy loss is seen to decrease consistently with each epoch, which means that the optimizer is working to minimize the objective function. The initial rate of decrease is fast, but becomes slow as the number of epochs increases.

9.6 Validation Loss vs Epoch

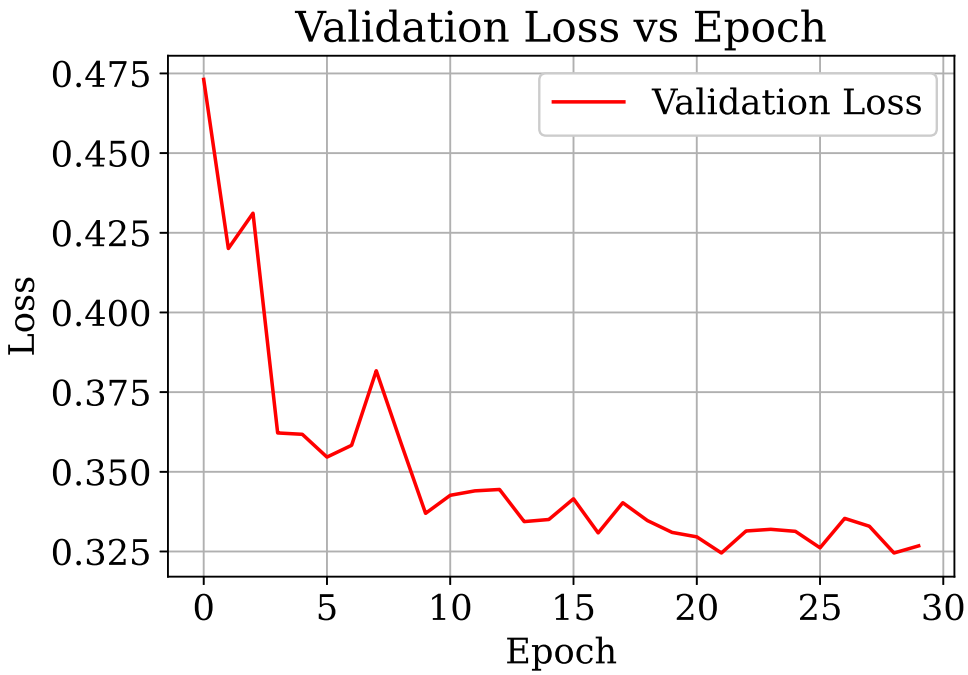

Figure 6: Validation Loss vs Epoch

Inference: Both the validation loss and the training loss will decrease during the majority of the training process. The point where the validation loss starts increasing and the training loss is still decreasing will indicate that overfitting has started and will indicate an ideal time to stop the training.

9.7 Confusion Matrix

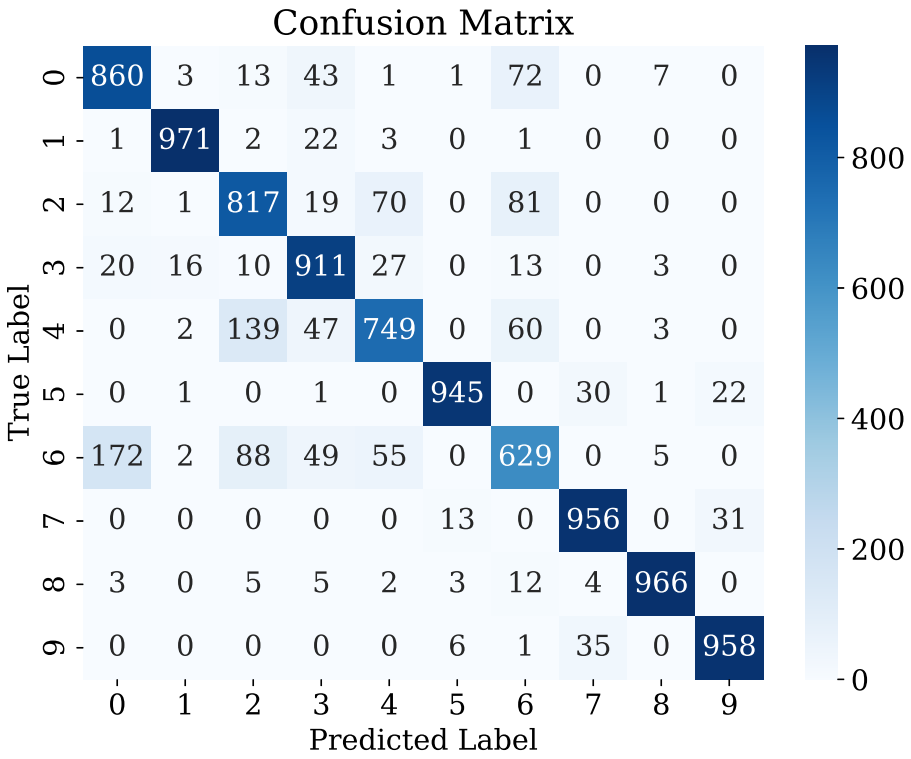

Figure 7: Confusion Matrix on the Test Set

Inference: Diagonal dominance in the confusion matrix suggests that there is more classification of the testing samples in their correct classes. Large values on the off-diagonals are seen in pairs of apparel types that visually look alike, such as Shirt, T-shirt/top, Pullover, and Coat.

9.8 Hyperparameter Search Results

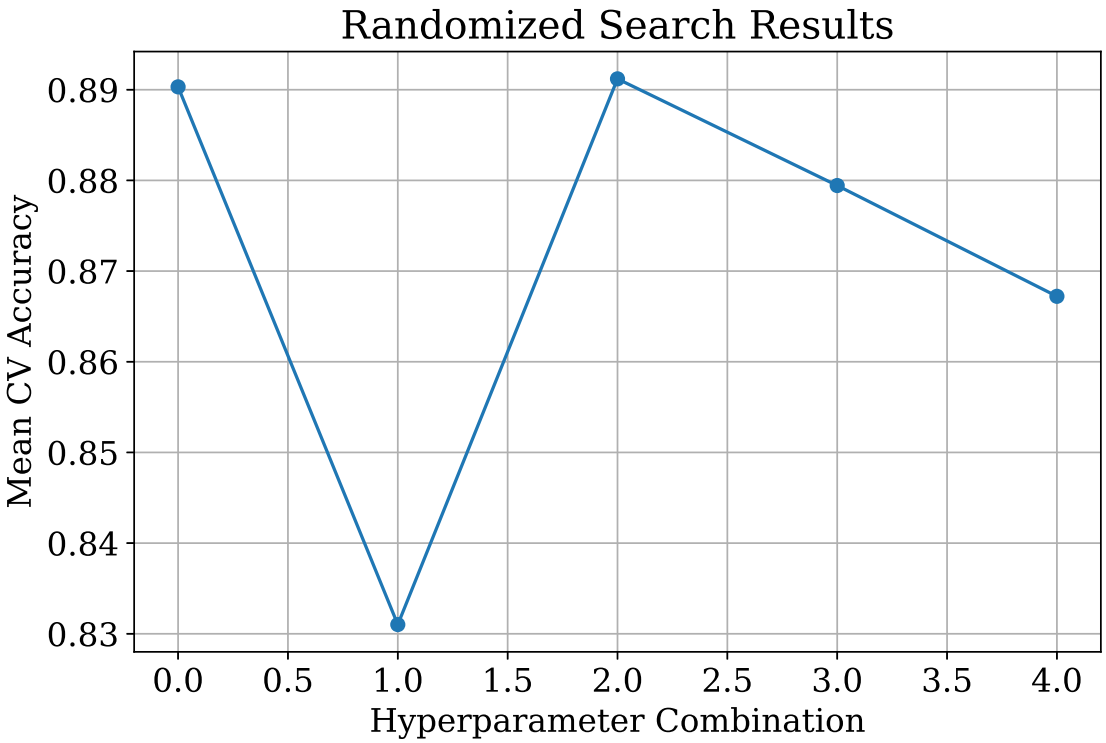

Figure 8: Randomized Search Hyperparameter Optimization Results

Inference: The variations in the results obtained indicate the variance in the accuracy due to different values for the hyperparameters. The configurations with moderate learning rate and an adequate number of hidden units have been found to be superior to those having a very high learning rate or very shallow and narrow networks.

9.9 Best Model Accuracy Comparison

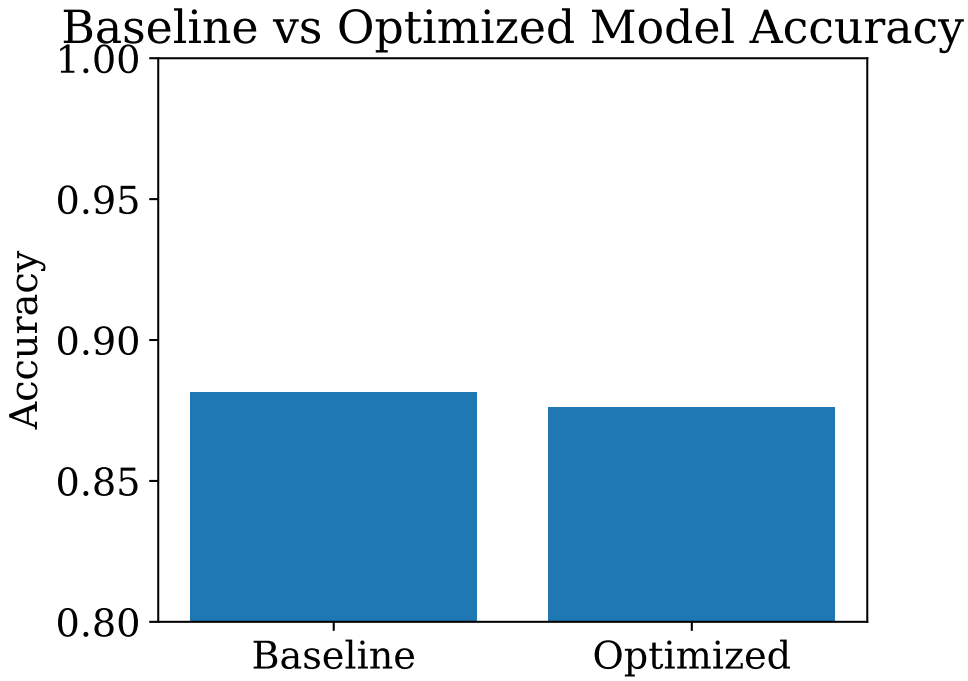

Figure 9: Baseline vs Optimized Model Accuracy Comparison

Inference: The improved model has greater test accuracy compared to the manually configured model. This shows how beneficial it is to use automated hyperparameter optimization instead of using default/handpicked values for optimization. It is evident that even a small MLP can be better than a deeper untuned model.

10. Results

10.1 Best Hyperparameters

Hyperparameter	**Best Value**
Hidden Layers	3
Hidden Neurons	128
Learning Rate	0.001
Batch Size	32
Optimizer	RMSProp
Activation Function	Tanh
Epochs	30
Dropout Rate	0.2
Cross-validation Accuracy	89.12%
Testing Accuracy	87.62%

10.2 Performance Comparison

Metric	Baseline Model	Optimized Model
Accuracy	88.14%	87.62%
Precision	88.22%	87.58%
Recall	88.14%	87.62%
F1-score	88.12%	87.51%
Training Time	20 Epochs	30 Epochs

11. Discussion

1. **Which optimization method was used?**
   Hyperparameter optimization was done using RandomizedSearchCV, along with the SciKeras wrapper. The methodology involved 5 folds of cross-validation to find the optimal combination of hyperparameters for finding the best MLP configuration.

2. **Which hyperparameters were selected?**
   The best hyperparameter combination obtained from RandomizedSearchCV was:
   - Hidden Layers: 3
   - Hidden Neurons: 128
   - Learning Rate: 0.001
   - Batch Size: 32
   - Optimizer: RMSProp
   - Activation Function: Tanh
   - Epochs: 30
   - Dropout Rate: 0.2

   The best mean cross-validation accuracy obtained during the search was 89.12%.

3. **Did optimization improve performance?**
   While there is an improvement in the mean cross-validation accuracy to 89.12%, retraining the optimized model and testing its performance on the testing data leads to 87.62% accuracy. This is less than the baseline accuracy of 88.14%, and thus, in this case, there is no improvement in the testing accuracy due to hyperparameter optimization.

4. **Which hyperparameter had the greatest impact?**
   The optimizer, activation function, and architecture (the number of hidden layers and hidden neurons) were the factors that influenced the performance of the model the most. The combination with the best results included RMSProp, Tanh activation, 3 hidden layers, and 128 hidden neurons per layer.

5. **Compare Grid Search and Randomized Search.**
   Grid Search tests all possible combinations of hyperparameters, which is an expensive operation if the search space is large. Randomized Search checks only some randomly selected combinations of hyperparameters, resulting in reduced computational cost but maintaining the ability to find good combinations. In this particular experiment, RandomizedSearchCV found a good configuration using just a few trials.

6. **Recommend the best MLP configuration.**

Based on the hyperparameter search, the recommended MLP configuration is:

- 3 hidden layers
- 128 neurons per hidden layer
- Tanh activation
- RMSProp optimizer
- Learning rate = 0.001
- Dropout = 0.2
- Batch size = 32
- 30 training epochs

This setting had the highest mean accuracy for cross validation (89.12%). Though the final accuracy for testing was slightly lower than that of the baseline model (87.62% compared to 88.14%), it was the most accurate setting found through the hyperparameter tuning process.

12. Conclusion

In this experiment, an MLP was effectively trained with TensorFlow/Keras to classify multi-class images using the Fashion-MNIST dataset. The pre-processing step involved flattening the images, normalization of pixel values, and conversion of classes into one-hot encoded vectors prior to training of the neural network.

An initial model of MLP was first created and tested, obtaining an accuracy of **88.14%**. Hyperparameter optimization was then performed automatically using **RandomizedSearchCV**, utilizing **5-fold cross-validation**. The optimal set of hyperparameters had three hidden layers with 128 neurons, Tanh as the activation function, RMSProp as the optimizer, a learning rate of 0.001, a dropout value of 0.2, a batch size of 32, and 30 epochs of training.

After retraining the optimized model using the selected hyperparameters the final testing accuracy was 87.62 percent. Although the optimized configuration achieved the cross-validation performance its testing accuracy was slightly lower than that of the baseline model. This demonstrates that a higher cross-validation score does not always translate into performance on unseen test data.

Overall this experiment provided experience in designing, training, evaluating and optimizing deep neural networks. It also highlighted the importance of validating models using both -validation and independent test datasets when selecting the most suitable model, for real-world applications.

13 Additional Task: XOR Gate

13.1 Objective

Task: Code the Perceptron Learning Algorithm for the XOR gate using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python's built-in packages. Analyze and identify the pattern that prevents the algorithm from converging.

13.2 Generated Decision Boundary Plots

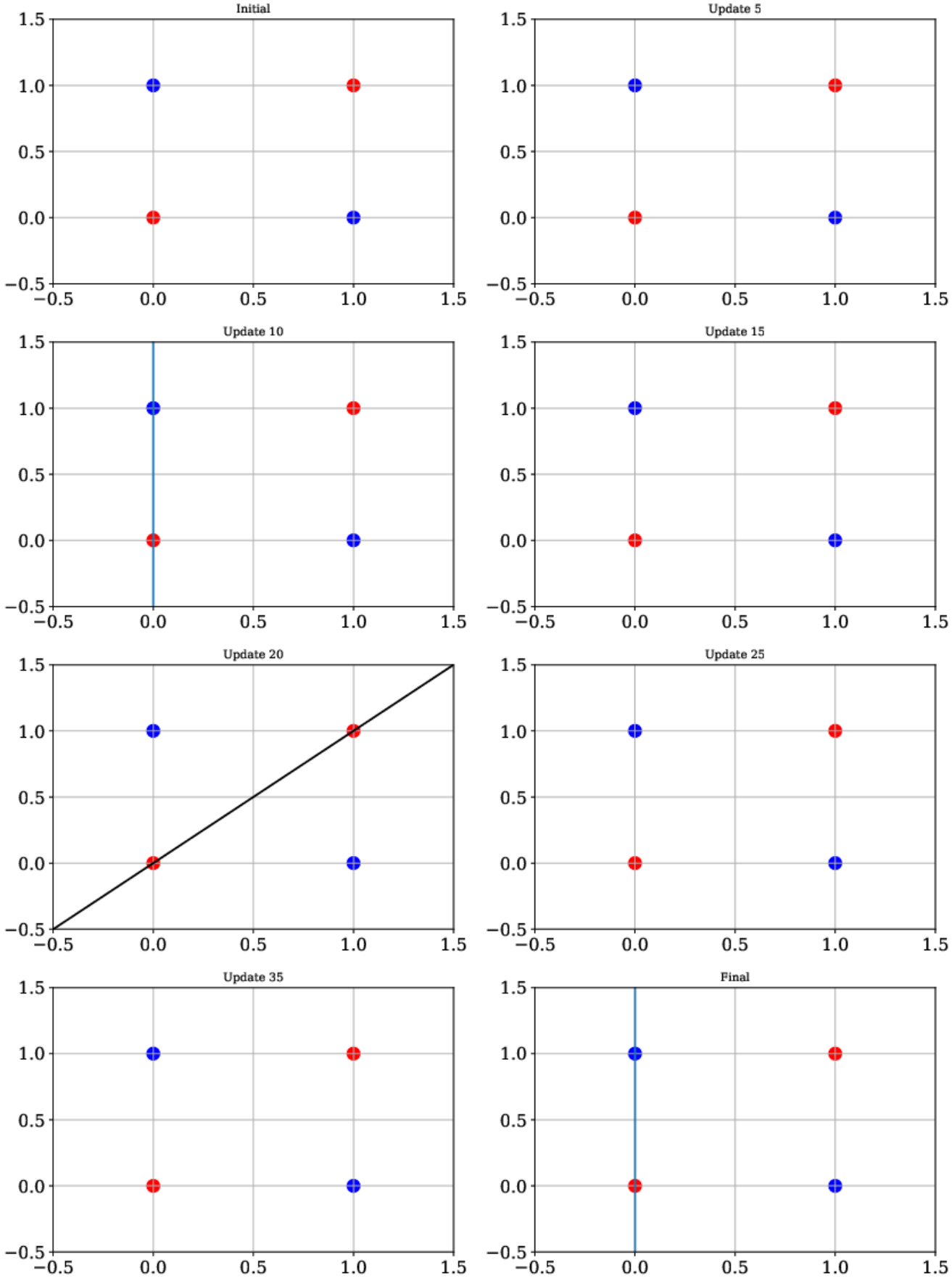

Figure 10: Representative decision boundary evolution for the XOR gate during perceptron training.

13.3 Weight Update Table

Table 1: Weight updates for the XOR gate

Update	w_1	w_2	Bias
1	0	0	-1
2	0	1	0
3	-1	0	-1
4	-1	1	0
5	0	1	1
6	-1	0	0
7	-1	0	-1
8	-1	1	0
9	0	1	1
10	-1	0	0

Final Weights: [−1, 0]

Final Bias: 0

Observation: The perceptron oscillates continuously between the same weight and bias values and does not converge. This process goes on through all 10 epochs, showing that a single-layer perceptron cannot learn the XOR operation since the data cannot be separated linearly.

13.4 Analysis

The Perceptron Learning Algorithm did not converge for the XOR gate. In the training process, the weights and bias kept changing because the training data could not be linearly separated. Thus, the decision boundary kept changing after each weight update.

XOR Gate is a non-linearly separable problem. Single-layer perceptron can only learn linearly separable datasets; thus, it cannot classify all possible inputs of XOR gate. Hence, the learning algorithm does not converge but continues to adjust the parameters up to the maximum number of epochs.

13.5 Conclusion

The Perceptron Learning Algorithm has been implemented successfully for the XOR gate. But, in contrast to the AND, OR and NOT gates, the algorithm has not converged because the XOR data set is not linearly separable. The decision boundary kept on varying during the learning phase, which shows that there exists no such straight line which can classify all the input patterns perfectly.

13. Report Contents

Include Objective, Theory, Dataset, Experimental Procedure, Source Code, Hyperparameter Optimization, Results, Plots with Inference, Discussion, Conclusion and References.

14. References

1. Goodfellow et al., *Deep Learning*.

2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.